Environment Configuration

Every environment variable and Secret Manager entry used by production PhysioGuidance — names, purposes, and which services consume them. No secret values are printed anywhere in this document.

PREPARED FOR
PhysioGuidance Client Team

DATE
2026-08-17

DEPLOYED COMMIT
d396b040

CLASSIFICATION
Confidential — names only, no values

PHYSIOGUIDANCE
Riyadh · Saudi Arabia
me-central2 · GCP Cloud Run

Table of contents

1. Storage principles
2. Secret Manager — the source of truth
3. Env vars per service — non-secret config
4. Secrets per service — mounted from Secret Manager
5. GitHub Actions env
6. Local development env
7. How secrets get from Secret Manager into a container
8. Adding a new secret
9. Adding a new env var
10. Component status matrix

---

1. Storage principles

- **Every production secret lives in GCP Secret Manager.** Every runtime credential the platform needs is fetched from Secret Manager at deploy time and mounted into the target container as an environment variable.
- **No secret is hard-coded in source code.** Verified against the repo — no hits for `sk_live`, `secret_key = "..."`, or similar literal patterns under `backends/src` or `frontend/src`.
- **`.env` files with real values stay off git.** `.gitignore` excludes `.env`, `.env.local`, `.env.production` at every workspace root.
- **`.env.example` files carry names and descriptions only** — no values.
- **This document lists names and purposes only** — never values.

---

2. Secret Manager — the source of truth

The `nelc-elearning-platform-479319` project holds every runtime secret used by production. Grouped by class, mirroring the exact set consumed by the deploy scripts under `infra/`:

Class	Count	Rotation cadence
Database + Redis credentials (`DATABASE_URL` , `STRAPI_DATABASE_PASSWORD` , `REDIS_HOST` , `REDIS_CA_CERT`)	4	On incident or team departure
Application-generated cryptographic keys (JWT × 2, PII encryption, reconciliation token, Strapi × 5)	9	12 months + on incident
Third-party API keys (Moyasar × 6, Tamara × 3, Tabby × 2, Zoom × 4, OpenAI, Resend, reCAPTCHA, NELC × 2, Sentry × 3)	22	12 months + vendor advisory + team departure
Mail address values held centrally (`MAIL_FROM` , `MAIL_REPLY_TO`)	2	Only when the sender/reply-to address changes
Public config held in Secret Manager for deploy-time injection (`NEXT_PUBLIC_RECAPTCHA_SITE_KEY` , `NEXT_PUBLIC_SENTRY_DSN` , `SENTRY_AUTH_TOKEN`)	3	On rotation of the corresponding private key

Total: 40 secrets used by the shipped deploy scripts. Full rotation procedures per secret in secrets-rotation-plan.md.

---

3. Env vars per service — non-secret config

Non-secret env vars are set via `gcloud run deploy --set-env-vars=...` inside each `infra/*/deploy.sh` script. Cloud Run injects the standard `PORT` at boot (do not set it explicitly).

3.1 physioguidance-api

Env var	Purpose
`NODE_ENV`	`production`
`API_PREFIX`	`api` (mounts routes under `/api/*`)
`LOG_LEVEL`	`warn` in prod (raise to `debug` for troubleshooting)
`CORS_ORIGIN`	Comma-separated list of allowed browser origins (custom domain + both Cloud Run URL formats for the web service)
`STORAGE_DRIVER`	`gcp` (uses Google Cloud Storage)
`GCP_PROJECT_ID`	GCP project identifier
`GCP_REGION`	`me-central2`
`STORAGE_BUCKET_PUBLIC`	GCS bucket for public course/blog assets (`physioguidance-public`)
`STORAGE_BUCKET_PRIVATE`	GCS bucket for private files (`physioguidance-private`)
`STORAGE_BUCKET_TRAINERS`	GCS bucket for trainer documents (`physioguidance-trainers`)
`BACKUPS_BUCKET`	GCS bucket for platform backups (`physioguidance-backups`)
`CLOUD_SQL_INSTANCE`	Cloud SQL instance name (`physioguidance-db-me`)
`DATABASE_NAME`	Postgres database name (`physioguidance`)
`APP_URL`	Public API URL (used in generated links)
`FRONTEND_URL`	Public web URL (used in email links)
`STRAPI_URL`	Public CMS URL (used by site-settings service for PDF branding)
`PAYMENT_CALLBACK_URL`	Frontend URL gateways redirect the buyer back to after payment
`PAYMENT_WEBHOOK_BASE_URL`	Public URL gateways POST webhooks to
`TASKS_ENABLED`	`true` in prod — enables Cloud Tasks event-driven scheduling
`TASKS_LOCATION`	`me-central2`
`TASKS_DISPATCH_URL`	Internal endpoint the task queue dispatches to
`TASKS_DISPATCHER_SA`	Service account that dispatches queued tasks
`TASKS_OIDC_AUDIENCE`	OIDC audience the dispatcher signs against; the receiver verifies it
`MOYASAR_MODE`	`test` or `live`; selects which Moyasar key set is active
`TAMARA_API_URL`	Tamara API base URL (production or sandbox)
`TABBY_API_URL`	Tabby API base URL

Env var	Purpose
`TABBY_MERCHANT_CODE`	Tabby-assigned merchant code
`TABBY_PUBLIC_KEY`	Tabby public key (safe to ship — used by backend for API signing)
`ZOOM_MOCK`	`false` when live Zoom credentials are mounted, `true` otherwise
`REDIS_PORT`	Memorystore port (`6378` for TLS)
`REDIS_TLS`	`true` (in-VPC Memorystore uses TLS)
`FFMPEG_ENABLED`	Toggles the video transcode pipeline
`AUTO_APPROVE_MATERIALS`	Auto-approve behavior for pipeline outputs
`NELC_LRS_ENABLED`	`true` — xAPI statements ship to NELC
`NELC_LRS_MODE`	Bootstrap default (`staging`); the admin UI overrides at runtime
`NELC_LRS_ENDPOINT_STAGING`	NELC staging LRS endpoint
`NELC_LRS_ENDPOINT_PROD`	NELC production LRS endpoint
`NELC_LRS_USERNAME`	Basic-auth user for NELC LRS (same across environments)
`NELC_PLATFORM_IDENTIFIER`	Platform URL sent as `context.platform` on every statement
`NELC_PLATFORM_NAME_EN`	English platform name for xAPI extension
`NELC_PLATFORM_NAME_AR`	Arabic platform name
`NELC_LMS_URL`	Base URL for canonical activity IDs
`SENTRY_ORG`	Sentry org slug (used by admin Error Monitoring tab)
`SENTRY_PROJECT`	Sentry project slug
`SENTRY_API_HOST`	Sentry API host (`de.sentry.io` for the EU region)
`SKIP_BULLMQ_WORKERS`	Set to `true` on api-web so BullMQ workers run only on the worker service
`EXTERNALIZED_CRONS`	Set to `true` on api-web when @Cron methods are executed by Cloud Run Jobs

3.2 physioguidance-api-worker

Same shape as api. Worker-specific overrides:

Env var	Purpose
`SKIP_CRONS`	`true` — @Cron scheduler is off on the worker (crons owned by api-web)

The `SKIP_BULLMQ_WORKERS` and `EXTERNALIZED_CRONS` toggles are absent on the worker so BullMQ consumers stay active there.

3.3 physioguidance-web

Runtime env vars set on the Cloud Run service:

Env var	Purpose
`NODE_ENV`	`production`
`BACKEND_API_URL`	Server-side URL used by Next.js server components to reach the API

All browser-side `NEXT_PUBLIC_*` values are baked in at Docker build time (see section 4.3). This keeps the runtime container free of any browser-exposed keys or URLs.

3.4 physioguidance-cms (Strapi)

Env var	Purpose
`NODE_ENV`	`production`
`DATABASE_CLIENT`	`postgres`
`DATABASE_HOST`	Cloud SQL socket path (`/cloudsql/<connection-name>`)
`DATABASE_PORT`	`5432`
`DATABASE_NAME`	Strapi database name (`strapi`)
`DATABASE_USERNAME`	Strapi DB user
`DATABASE_SSL`	`false` — traffic is local IPC via the Cloud Run Cloud SQL socket
`GCS_BUCKET_NAME`	GCS bucket for Strapi uploads (`physioguidance-cms`)
`GCS_BASE_PATH`	Optional path prefix inside the bucket
`GCS_PUBLIC_FILES`	`true` — uploads are public by default
`GCS_UNIFORM_ACCESS`	`true` — bucket has uniform bucket-level access

Cloud Run injects `HOST` and `PORT` at boot; Strapi's `server.ts` picks them up automatically.

3.5 physioguidance-malware-scanner

Env var	Purpose
`QUARANTINE_BUCKET`	Destination bucket for infected files (`physioguidance-quarantine`)
`WATCHED_BUCKETS`	Comma-separated list of buckets Eventarc triggers watch (`physioguidance-private,physioguidance-public,physioguidance-trainers`)

4. Secrets per service — mounted from Secret Manager

Each service's `infra/*/deploy.sh` uses `--set-secrets=<envVarName>=<secretName>:latest` to mount specific secrets. Names below reflect the Secret Manager secret name; the value is never present in code, CI logs, or this document.

4.1 physioguidance-api

Env var name inside container	Secret Manager secret	Purpose
`DATABASE_URL`	`DATABASE_URL`	Postgres connection string (Unix-socket form via `/cloudsql/`)
`JWT_SECRET`	`JWT_SECRET`	Signs access tokens
`JWT_REFRESH_SECRET`	`JWT_REFRESH_SECRET`	Signs refresh tokens
`PII_ENCRYPTION_KEY`	`PII_ENCRYPTION_KEY`	AES-256-GCM key for PII field encryption on the users table
`RECAPTCHA_SECRET_KEY`	`RECAPTCHA_SECRET_KEY`	reCAPTCHA v2 server-side verification for `/auth/login` and `/auth/register`
`RECONCILIATION_CRON_TOKEN`	`RECONCILIATION_CRON_TOKEN`	Belt-and-braces `X-Cron-Token` header on payment reconciliation endpoints
`RESEND_API_KEY`	`RESEND_API_KEY`	Primary transactional email delivery
`MAIL_FROM`	`MAIL_FROM`	Sender email address for outbound email
`MAIL_REPLY_TO`	`MAIL_REPLY_TO`	Reply-to email address
`REDIS_HOST`	`REDIS_HOST`	Memorystore host (VPC private IP)
`REDIS_CA_CERT`	`REDIS_CA_CERT`	CA bundle for verified Memorystore TLS
`OPENAI_API_KEY`	`OPENAI_API_KEY`	Whisper transcription for video captions
`MOYASAR_SECRET_KEY_TEST`	`MOYASAR_SECRET_KEY_TEST`	Moyasar secret key (test mode)
`MOYASAR_PUBLISHABLE_KEY_TEST`	`MOYASAR_PUBLISHABLE_KEY_TEST`	Moyasar publishable key (test mode)
`MOYASAR_WEBHOOK_SECRET_TEST`	`MOYASAR_WEBHOOK_SECRET_TEST`	Moyasar webhook signature verification (test mode)

Env var name inside container	Secret Manager secret	Purpose
`MOYASAR_SECRET_KEY_LIVE`	`MOYASAR_SECRET_KEY_LIVE`	Moyasar secret key (live mode)
`MOYASAR_PUBLISHABLE_KEY_LIVE`	`MOYASAR_PUBLISHABLE_KEY_LIVE`	Moyasar publishable key (live mode)
`MOYASAR_WEBHOOK_SECRET_LIVE`	`MOYASAR_WEBHOOK_SECRET_LIVE`	Moyasar webhook signature verification (live mode)
`TAMARA_API_TOKEN`	`TAMARA_API_TOKEN`	Tamara API JWT token
`TAMARA_NOTIFICATION_TOKEN`	`TAMARA_NOTIFICATION_TOKEN`	Tamara webhook notification token
`TAMARA_PUBLIC_KEY`	`TAMARA_PUBLIC_KEY`	Tamara public key (webhook signature verification)
`TABBY_SECRET_KEY`	`TABBY_SECRET_KEY`	Tabby API secret
`TABBY_WEBHOOK_SECRET`	`TABBY_WEBHOOK_SECRET`	Tabby webhook signature
`ZOOM_ACCOUNT_ID`	`ZOOM_ACCOUNT_ID`	Zoom S2S OAuth account ID
`ZOOM_CLIENT_ID`	`ZOOM_CLIENT_ID`	Zoom S2S OAuth client ID
`ZOOM_CLIENT_SECRET`	`ZOOM_CLIENT_SECRET`	Zoom S2S OAuth client secret
`ZOOM_WEBHOOK_SECRET_TOKEN`	`ZOOM_WEBHOOK_SECRET_TOKEN`	Zoom webhook secret
`SENTRY_DSN`	`SENTRY_DSN_BACKEND`	Backend error monitoring DSN
`SENTRY_API_TOKEN`	`SENTRY_API_TOKEN`	Sentry REST API access for the admin Error Monitoring tab
`SENTRY_WEBHOOK_SECRET`	`SENTRY_WEBHOOK_SECRET`	HMAC signature verification on inbound Sentry alert webhooks
`NELC_LRS_PASSWORD`	`NELC_LRS_PASSWORD`	NELC LRS basic-auth password (staging)
`NELC_LRS_PASSWORD_PROD`	`NELC_LRS_PASSWORD_PROD`	NELC LRS basic-auth password (production)

4.2 physioguidance-api-worker

Mirrors the api set exactly, except:

- **Included:** everything api-web needs to boot the same code path — DB, JWT, PII key, Redis, OpenAI, Resend, Mail addresses, Sentry DSN + API token, Zoom, all Moyasar keys, Tamara, Tabby, NELC passwords.
- **Not mounted:** `RECAPTCHA_SECRET_KEY` (worker serves no HTTP), `RECONCILIATION_CRON_TOKEN` (cron endpoints live on api-web), `SENTRY_WEBHOOK_SECRET` (inbound webhook endpoint lives on api-web).

Every secret mounted on the worker also appears in `infra/backend/deploy.sh`; the two scripts are kept in sync manually — adding a new API secret means adding it to both.

4.3 physioguidance-web

Frontend secrets are pulled at deploy time by `infra/frontend/deploy.sh` and passed to `docker build --build-arg` so they bake into the immutable image:

Build arg	Secret Manager secret	Purpose
`NEXT_PUBLIC_RECAPTCHA_SITE_KEY`	`NEXT_PUBLIC_RECAPTCHA_SITE_KEY`	reCAPTCHA v2 site key (safe to ship — public by design)
`NEXT_PUBLIC_SENTRY_DSN`	`NEXT_PUBLIC_SENTRY_DSN`	Frontend Sentry DSN (public per Sentry docs)
`SENTRY_AUTH_TOKEN`	`SENTRY_AUTH_TOKEN`	CI-only token for source-map upload to Sentry (stripped from browser bundle)

Additional build-time env vars derived by the deploy script (not stored in Secret Manager — they are constants or come from `infra/config.sh`):

Build arg	Source	Purpose
`NEXT_PUBLIC_API_URL`	Resolved backend URL + `/api`	Base URL the browser calls
`NEXT_PUBLIC_STRAPI_URL`	Resolved Strapi URL	Base URL for CMS-served assets
`NEXT_PUBLIC_APP_ENV`	`production`	Environment label surfaced to Sentry
`NEXT_PUBLIC_SENTRY_RELEASE`	Deploy timestamp	Pins each deploy to its source-map bundle
`NEXT_PUBLIC_TABBY_PUBLIC_KEY`	Constant	Tabby BNPL widget public key
`NEXT_PUBLIC_TABBY_MERCHANT_CODE`	Constant	Tabby BNPL merchant code
`NEXT_PUBLIC_TAMARA_PUBLIC_KEY`	Constant	Tamara BNPL widget public key
`NEXT_PUBLIC_TAMARA_CDN_URL`	Constant	Tamara widget CDN base URL
`SENTRY_ORG`	`physioguidance`	Sentry org slug for source-map upload
`SENTRY_PROJECT`	`physioguidance-web`	Sentry project slug for source-map upload

4.4 physioguidance-cms (Strapi)

Env var name inside container	Secret Manager secret	Purpose
`APP_KEYS`	`STRAPI_APP_KEYS`	Session-signing key rotation set
`API_TOKEN_SALT`	`STRAPI_API_TOKEN_SALT`	Salts Strapi API tokens
`ADMIN_JWT_SECRET`	`STRAPI_ADMIN_JWT_SECRET`	Signs admin panel JWTs
`TRANSFER_TOKEN_SALT`	`STRAPI_TRANSFER_TOKEN_SALT`	Salts Strapi data-transfer tokens
`JWT_SECRET`	`STRAPI_JWT_SECRET`	Signs public-API JWTs
`DATABASE_PASSWORD`	`STRAPI_DATABASE_PASSWORD`	Strapi Postgres user password

The Strapi Cloud SQL connection is composed from `DATABASE_HOST` (the Cloud SQL socket), `DATABASE_PORT`, `DATABASE_NAME`, `DATABASE_USERNAME` (env vars) plus `DATABASE_PASSWORD` (secret). There is no single `DATABASE_URL` secret for Strapi.

4.5 physioguidance-malware-scanner

No Secret Manager mounts. The scanner uses its runtime service account (`malware-runtime@…`) for GCS access; ClamAV signature updates come from ClamAV's own signature servers at container build

time (`freshclam` runs inside the Dockerfile).

---

5. GitHub Actions env

Four GitHub Actions secrets exist, all of them infra pointers rather than runtime credentials:

GitHub secret	Purpose
`WIF_PROVIDER`	Full Workload Identity Federation provider resource name
`WIF_SERVICE_ACCOUNT`	Deploy service account email
`GCP_PROJECT_ID`	`nelc-elearning-platform-479319`
`GCP_REGION`	`me-central2`

Every runtime app secret is pulled at deploy time via `gcloud secrets versions access` inside the deploy script — GitHub Actions itself holds only WIF pointers.

---

6. Local development env

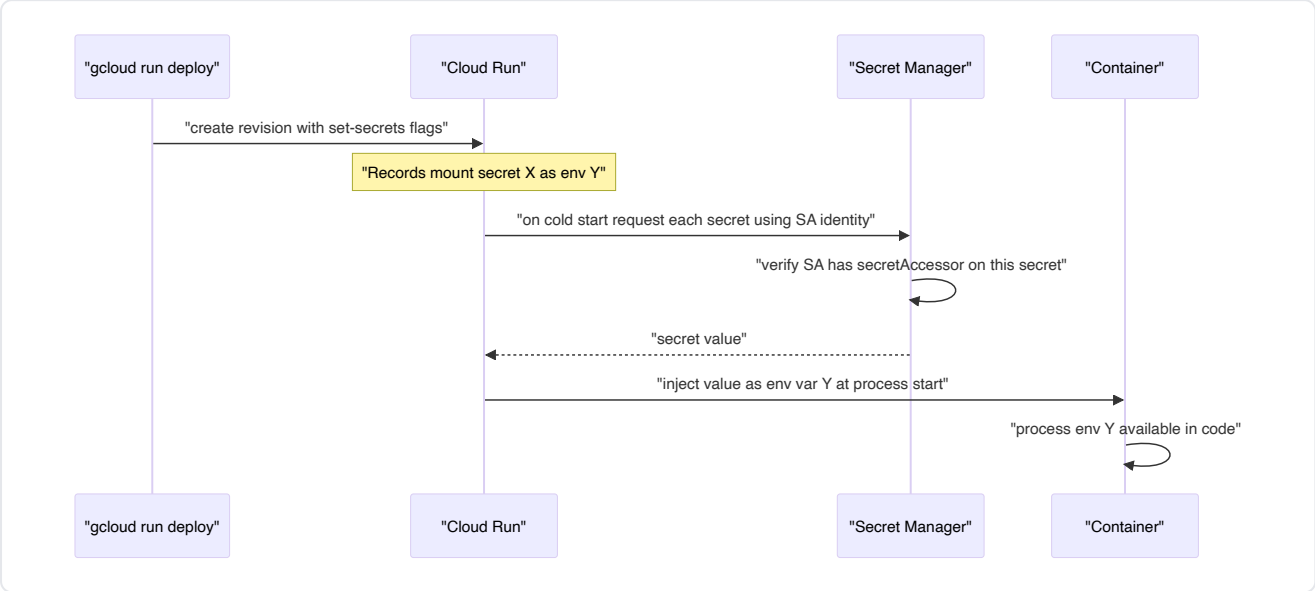
Each workspace has an `.env.example` file with names and descriptions only. Developers copy it to `.env.local` and fill in dev values.

Workspace	Example file
backends	`backends/.env.example`
strapi	`strapi/.env.example`
frontend	Local overrides via `.env.local` (checked into `.gitignore`)

Local dev uses reasonable placeholders — Moyasar test keys, sandbox Zoom app, local Postgres URL, Mailpit SMTP — all documented in the example files. Production secrets never leave Secret Manager.

---

7. How secrets get from Secret Manager into a container



No file on disk. Secrets live in the container's process env memory only — never written to a file, never logged, never included in the container image.

---

8. Adding a new secret

1. Generate the value locally: `openssl rand -base64 32 | tr -d '\n' | pbcopy`
2. Create the secret in Secret Manager: `gcloud secrets create MY_NEW_SECRET --data-file=-`
3. Grant the consuming service's runtime SA access: `gcloud secrets add-iam-policy-binding MY_NEW_SECRET --member=serviceAccount:<runtime-sa> --role=roles/secretmanager.secretAccessor`
4. Add the `--set-secrets=MY_NEW_SECRET=MY_NEW_SECRET:latest` line to the relevant `infra/*/deploy.sh` (add to both api and worker deploy scripts if it applies to both)
5. Redeploy the service
6. Document the secret in secrets-rotation-plan.md with its rotation procedure

---

9. Adding a new env var

Non-secret only:

1. Add to the `--set-env-vars=` list in the relevant `infra/*/deploy.sh`

2. Add to `backends/.env.example` (or the frontend/strapi equivalent) with name + description + reasonable default
3. Read it in code via `ConfigService` (backend) or `process.env.NAME` (frontend/strapi)
4. Redeploy

URLs and constants are fine in `.env.example`; real secret values never are.

---

10. Component status matrix

Using Amna's four-category vocabulary.

Item	Status	Notes
All production secrets in GCP Secret Manager	Implemented	40 secrets used by shipped deploy scripts; verifiable via `gcloud secrets list`
Every runtime secret sourced from Secret Manager	Implemented	Deploy scripts pull each secret with `gcloud secrets versions access` at deploy time
No secret hard-coded in source	Implemented	Convention + grep-verified against `sk_live` and similar literals
`.env` files with real values stay off git	Implemented	`.gitignore` covers `.env`, `.env.local`, `.env.production` at every workspace root
`.env.example` contains names + descriptions only	Implemented	Verified per workspace
Another authorized developer can deploy using client-owned GCP access	Implemented	See cicd-deployment.md §10
Credential/access rotation plan documented	Implemented	secrets-rotation-plan.md — full per-secret runbook
Per-service runtime SA scoped to only the secrets it consumes	Implemented	api-runtime, worker-runtime, web-runtime, strapi-runtime, malware-runtime each bind only their own secret set
Frontend browser-facing values baked at build time only	Implemented	Runtime container carries only `NODE_ENV` + `BACKEND_API_URL`; all `NEXT_PUBLIC_*` values baked into the image

---

11. Where to look next

- **Per-secret rotation procedures** → secrets-rotation-plan.md
- **How CI reads build-time secrets** → cicd-deployment.md §5
- **How PII encryption uses** `PII_ENCRYPTION_KEY` → security-architecture.md §10